

I'm doing a lab where I fix and extend a terminal Connect Four game (Python). It has four main codes: ai.py, board.py, game.py, and main.py. I'll give you one task at a time. Don't rewrite the whole project, keep the existing structure, keep all state in memory, and add no dependencies or persistent storage. Explain each change. Task 1: Complete win detection. Make win detection recognise every four-in-a-row direction, including both diagonals, without breaking horizontal and vertical wins. Done when all four directions are detected and shorter sequences do not count. Here is board.py:

Python

```
ROWS, COLS = 6, 7

class Board:
    def __init__(self):
        self.grid = [["."] * COLS for _ in range(ROWS)]

    def drop(self, col, token):
        if not 0 <= col < COLS:
            return None
        for r in range(ROWS - 1, -1, -1):
            if self.grid[r][col] == ".":
                self.grid[r][col] = token
                return r
```

Also give me a small test script that checks horizontal, vertical, both diagonals, and that 3 in a row does not count.

You only need to make a small change to `board.py`. Your existing structure is correct; the issue is that `winner()` currently checks only horizontal and vertical directions, not diagonals.

1. Update `board.py`

Replace your existing `winner()` method with this version. **Leave the rest of `board.py` unchanged.**

Writing



```

python
def winner(self, token):
    # Horizontal, vertical, diagonal down-right, diagonal down-left
    directions = [
        (0, 1),
        (1, 0),
        (1, 1),
        (1, -1)
    ]

    for r in range(ROWS):
        for c in range(COLS):
            if self.grid[r][c] != token:
                continue

            for dr, dc in directions:
                if all(
                    0 <= r + i * dr < ROWS and
                    0 <= c + i * dc < COLS and
                    self.grid[r + i * dr][c + i * dc] == token
                    for i in range(4)
                ):
                    return True

    return False

```

What changed?

The only functional change is adding the two diagonal directions.

Direction	(dr, dc)	What it checks
Horizontal →	(0, 1)	Four tokens across a row

Vertical ↓	(1, 0)	Four tokens down a column
Diagonal ↘	(1, 1)	Four tokens sloping down-right
Diagonal ↙	(1, -1)	Four tokens sloping down-left

The existing `all()` checks all four positions, starting from the current cell. It also checks the board boundaries before accessing each position, preventing out-of-range errors.

Why three in a row won't count: the method still requires all four positions to contain the specified token. If even one position is empty or contains another token, `all()` returns `False`.

2. Create a test script

Create a new file named `testboard.py` in the same folder as `board.py`.

Writing



```

python
import unittest
from board import Board

def make_board(cells, token="X"):
    """Place tokens at the specified (row, column) positions."""
    b = Board()
    for r, c in cells:
        b.grid[r][c] = token
    return b

class TestWinDetection(unittest.TestCase):

    def test_horizontal_win(self):
        b = make_board([(2, 1), (2, 2), (2, 3), (2, 4)])
        self.assertTrue(b.winner("X"))

    def test_vertical_win(self):
        b = make_board([(1, 3), (2, 3), (3, 3), (4, 3)])

```

```
self.assertTrue(b.winner("X"))

def test_diagonal_down_right_win(self):
    b = make_board([(0, 0), (1, 1), (2, 2), (3, 3)])
    self.assertTrue(b.winner("X"))

def test_diagonal_down_left_win(self):
    b = make_board([(0, 3), (1, 2), (2, 1), (3, 0)])
    self.assertTrue(b.winner("X"))

def test_three_in_a_row_is_not_a_win(self):
    b = make_board([(2, 1), (2, 2), (2, 3)])
    self.assertFalse(b.winner("X"))

if __name__ == "__main__":
    unittest.main()
'''
```

These tests cover all four winning directions and the three-in-a-row case. The test boards are created in memory, so nothing is saved to disk.

3. Run the tests

Open your terminal in the project folder and run:

</> Bash

```
python -m unittest testboard.py -v
```

If your system uses the `python3` command instead, run:

</> Bash

```
python3 -m unittest testboard.py -v
```

Expected result:

</> Plain text

```
test_diagonal_down_left_win ... ok
test_diagonal_down_right_win ... ok
test_horizontal_win ... ok
test_three_in_a_row_is_not_a_win ... ok
test_vertical_win ... ok
```

Ran 5 tests

OK

4. Task 1 completion checklist

- ✓ Horizontal wins still work.
- ✓ Vertical wins still work.
- ✓ Down-right diagonal wins are detected.
- ✓ Down-left diagonal wins are detected.
- ✓ Three consecutive tokens do not count as a win.
- ✓ No dependencies or persistent storage added.
- ✓ Existing project structure preserved.

One small detail: the tests above check exactly three tokens in a row. Your implementation also correctly detects a win when there are **four or more consecutive tokens**, since it checks for four matching positions rather than requiring exactly four.

Now Task 2: Complete game termination. Handle draws, full columns, invalid moves, and quitting consistently. A winning move must stop the game before another turn is requested. No invalid move may change the board, and every terminal state must be clear. Current behaviour in game.py: every rejected move prints "Column unavailable." (even for out-of-range numbers like 0 or 99), non-numbers print "Enter a column number.", and the game silently ends if the AI has no move.

Please give distinct, clear messages for non-numbers, out-of-range columns, and full columns, and a clear message for each end state (win, draw, quit). Keep the structure of game.py, don't change the

board state on invalid input, and explain your changes. Also give me a quick way to test a draw without playing a whole game.

Here's the old game.py: from board import Board
from ai import AI
class Game:
def init(self):
self.board = Board()
self.ai = AI()
self.turn = "X"

```
def run(self):
    print("Connect Four – you are X.")
    while True:
        self.board.print()
        if self.turn == "X":
            raw = input("Column (1-7), or q: ").strip()
            if raw == "q":
                return
            try:
                col = int(raw) - 1
            except ValueError:
                print("Enter a column number.")
                continue
        else:
```

For Task 2, you can keep the existing `Game` class and `run()` loop structure. The main fix is to **validate a move before modifying the board**, distinguish the different reasons a move can fail, and ensure that every ending returns from the game loop immediately.

The important cases are:

- **Non-number:** input such as `abc` → ask for a column number.
- **Out of range:** `0`, `8`, or `99` → tell the player to enter a column from 1 to 7.
- **Full column:** a valid column that has no empty spaces → reject the move without changing the board.
- **Quit:** `q` → print a clear quit message and end the game.

- **Win:** print the winning board and winner message, then stop before requesting another move.
- **Draw:** when the board is full and nobody has won, print a draw message and stop.
- **AI has no move:** handle this explicitly rather than silently ending.

1. Replace `game.py` with this version

This keeps your existing imports, class, attributes, and `run()` method. No changes to `board.py`, `ai.py`, or `main.py` are required for this task.

Writing



```
```python
from board import Board
from ai import AI

class Game:
 def __init__(self):
 self.board = Board()
 self.ai = AI()
 self.turn = "X"

 def run(self):
 print("Connect Four — you are X.")

 while True:
 self.board.print()

 # Handle a full board before requesting another move.
 if self.board.full():
 print("Draw! The board is full.")
 return

 if self.turn == "X":
 raw = input("Column (1-7), or q: ").strip().lower()

 if raw == "q":
 print("Game quit. Thanks for playing!")
```

```
 return

 try:
 col = int(raw) - 1
 except ValueError:
 print("Invalid input. Enter a column number from 1 to 7, or q to quit.")
 continue

 # Check the range before attempting to drop a token.
 if not 0 <= col < 7:
 print("Invalid column. Choose a number from 1 to 7.")
 continue

 # Reject full columns without changing the board.
 if self.board.grid[0][col] != ".":
 print(f"Column {col + 1} is full. Choose another column.")
 continue

 else:
 col = self.ai.choose_column(self.board)

 # Handle an AI that cannot find a move.
 if col is None:
 if self.board.full():
 print("Draw! The board is full.")
 else:
 print("AI could not find a move. Game ended.")
 return

 # Validate the AI's selected column too.
 if not isinstance(col, int) or not 0 <= col < 7:
 print("AI returned an invalid column. Game ended.")
 return

 if self.board.grid[0][col] != ".":
 print(f"AI selected full column {col + 1}. Game ended.")
 return
```

```

Only valid moves reach this point.
row = self.board.drop(col, self.turn)

Defensive check in case the move is rejected.
if row is None:
 print("Move rejected. Please try again.")
 if self.turn == "O":
 print("AI could not complete its move. Game ended.")
 return
 continue

Check for a win immediately after the move.
if self.board.winner(self.turn):
 self.board.print()
 if self.turn == "X":
 print("You win!")
 else:
 print("AI wins!")
 return

Check for a draw after checking for a win.
if self.board.full():
 self.board.print()
 print("Draw! The board is full.")
 return

Switch turns only when the game is still active.
self.turn = "O" if self.turn == "X" else "X"

```

...

## 2. Explanation of the changes

### A. Separate invalid-input messages

Input or situation	Message
--------------------	---------

hello , empty input, 2.5

Invalid input — enter a column number

0 , 8 , 99 , -1

Invalid column — choose 1 to 7

Column 3 is full

Column 3 is full — choose another

q

Game quit — thanks for playing

The `continue` statement restarts the loop for rejected human moves. Since `drop()` has not been called yet, the board remains unchanged.

## B. Winning moves terminate immediately

The order matters:

1. Drop the token.
2. Check whether that move wins.
3. Check whether the board is full.
4. Switch turns only if the game is still active.

This ensures a winning move ends the game before another player or the AI gets a turn.

## C. Draws are checked explicitly

The board is checked for fullness before requesting another move, and again after each successful move. The win check comes first after a move, so a winning move on the last available space is reported as a win rather than a draw.

## D. AI failures are no longer silent

If the AI returns `None`, selects an invalid column, or selects a full column, the game displays a clear message and terminates. This prevents an endless loop or a silent exit.

## 3. Test a draw without playing a whole game

Create a file called `test_draw.py` in the same folder as your project.

This fills the board directly in memory with a pattern that contains no four consecutive matching tokens in any direction. It then starts the game, which should immediately recognise the draw without asking for input.

Writing



```
```python
from game import Game

game = Game()

# Fill the board with a pattern that has no four-in-a-row.
for r in range(6):
    for c in range(7):
        game.board.grid[r][c] = (
            "X" if (r + 2 * c) % 4 < 2 else "O"
        )

game.run()
```
```

Run it in your terminal:

**</> Bash**

```
python test_draw.py
```

Expected ending:

**</> Plain text**

```
Draw! The board is full.
```

The board will print before the message. No tokens are played, and no files are created by the test.

## 4. Quick manual tests

Run your regular game using your usual command, then try these cases:

**Manual test checklist**

0 of 5 tested

- ☐ **abc**  
Invalid input message; board unchanged
- ☐ **0**  
Out-of-range message; board unchanged
- ☐ **99**  
Out-of-range message; board unchanged
- ☐ **q**  
Quit message and immediate exit
- ☐ **A full column**  
Full-column message; board unchanged

For the full-column case, enter the same column seven times only if the game continues to allow that column to fill; once it is full, the next attempt should be rejected. The AI may also play between your turns.

**Task 2 completion criteria:** distinct validation messages, no board changes on invalid human input, clear win/draw/quit messages, immediate termination after a win, and explicit handling of AI move failures.

Add meaningful decision-making. The AI should take an immediate winning move when one exists and block an immediate player win when necessary. Handle full columns safely.

Done when: the AI never selects an illegal column and responds correctly to one-move threats.

Current behaviour in ai.py: the AI just picks a random legal column, so it ignores winning moves and doesn't block my four-in-a-rows. Please make it check for its own winning move first, then check for a move that blocks my win, and otherwise fall back to a sensible move (e.g. prefer the centre, then random). Testing a hypothetical move must not permanently change the real board. Keep the existing `choose_column(board, me="O", opponent="X")` signature so game.py doesn't need to change, and explain your changes. Also give me a quick way to test an AI win and an AI block without playing a whole game.

Here's the current ai.py: `import random`

```

class AI:
 def choose_column(self, board, me="O", opponent="X"):
 # Starter AI is deliberately basic; students improve it in Task 3.
 legal = [c for c in range(7) if board.grid[0][c] == "."]
 return random.choice(legal) if legal else None

Here's the current board.py:
def winner(self, token):
 # Horizontal, vertical, diagonal down-right, diagonal down-left
 directions = [
 (0, 1),
 (1, 0),
 (1, 1),
 (1, -1)
]

 for r in range(ROWS):
 for c in range(COLS):
 if self.grid[r][c] != token:
 continue

 for dr, dc in directions:
 if all(
 0 <= r + i * dr < ROWS and
 0 <= c + i * dc < COLS and
 self.grid[r + i * dr][c + i * dc] == token
 for i in range(4)
):
 return True

 return False

```

For Task 3, you only need to update `ai.py`. We'll keep the existing `choose_column(board, me="O", opponent="X")` signature, so your `game.py` does not need to change.

The AI will follow this priority:

1. **Win immediately:** If the AI can complete four in a row, play that column.
2. **Block immediately:** If the player can win on their next move, play that column to block them.

3. **Prefer the centre:** If neither applies, prefer the centre columns because they offer more potential winning combinations.
4. **Random fallback:** Choose randomly among equally preferred legal columns.

Every candidate move will be tested temporarily and then undone, so testing moves won't permanently modify the board.

## 1. Replace `ai.py`

Replace the current contents with this code:

Writing



```
```python
import random

class AI:
    def choose_column(self, board, me="O", opponent="X"):
        # Find columns that can legally accept a token.
        legal = [
            c for c in range(7)
            if board.grid[0][c] == "."
        ]

        # No legal moves are available.
        if not legal:
            return None

        # 1. Take an immediate winning move.
        for col in legal:
            row = board.drop(col, me)

            if row is not None:
                wins = board.winner(me)
                board.grid[row][col] = "."

                if wins:
```

```

        return col

# 2. Block an immediate opponent win.
for col in legal:
    row = board.drop(col, opponent)

    if row is not None:
        wins = board.winner(opponent)
        board.grid[row][col] = "."

    if wins:
        return col

# 3. Prefer the centre column, then nearby columns.
centre = 3
best_distance = min(abs(c - centre) for c in legal)
preferred = [
    c for c in legal
    if abs(c - centre) == best_distance
]

# Randomly choose among equally preferred columns.
return random.choice(preferred)
...

```

2. How the changes work

Step 1: Find legal columns

</> Python

```

legal = [
    c for c in range(7)
    if board.grid[0][c] == "."
]

```

Only columns whose top cell is empty are considered. If all columns are full, the AI returns `None` instead of trying to select an illegal move.

Step 2: Check for an AI win

For each legal column, the AI temporarily drops its own token and calls `board.winner(me)`.

If that move wins, it remembers the column and returns it. Otherwise, it undoes the hypothetical move before testing the next column.

Step 3: Block the player

If the AI has no immediate win, it repeats the same process using the opponent's token. If the opponent would win by playing in a particular column, the AI chooses that column to block the threat.

Step 4: Choose a sensible fallback

If neither a win nor a block is available, the AI prefers the legal column closest to the centre (column 4 for the player, represented by index `3` in Python). If multiple columns are equally close, it picks randomly among them.

Why hypothetical moves don't change the real board

The important part is:

`</>` Python

```
row = board.drop(col, me)
wins = board.winner(me)
board.grid[row][col] = "."
```

`drop()` returns the row where the token was placed. After checking for a win, the AI restores that exact cell to `"."`. The same process is used for the opponent's hypothetical move.

This assumes `board.drop()` places a token in the lowest available cell, as in your current `board.py`.

3. Quick tests: AI win, AI block, and full board

Create `test_ai.py` in the same folder as `ai.py` and `board.py`.

Writing



```
```python
from board import Board
from ai import AI

ai = AI()

def make_board(token, columns):
 board = Board()

 # Place tokens on the bottom row for a simple test.
 for col in columns:
 board.grid[5][col] = token

 return board

Test 1: AI should win by completing four O tokens.
board = make_board("O", [0, 1, 2])
before = [row[:] for row in board.grid]

move = ai.choose_column(board)
assert move == 3, f"Expected column 4 to win, got {move + 1 if move is not None else None}"
assert board.grid == before, "AI changed the board while testing!"
print("PASS: AI takes an immediate winning move.")

Test 2: AI should block four X tokens.
board = make_board("X", [0, 1, 2])
before = [row[:] for row in board.grid]

move = ai.choose_column(board)
assert move == 3, f"Expected column 4 to block, got {move + 1 if move is not None else None}"
assert board.grid == before, "AI changed the board while testing!"
print("PASS: AI blocks an immediate opponent win.")
```

```
Test 3: AI returns None when every column is full.
board = Board()
for col in range(7):
 board.grid[0][col] = "X"

assert ai.choose_column(board) is None
print("PASS: AI handles a board with no legal columns.")

print("All AI tests passed!")
'''
```

Run the tests with:

**</> Bash**

```
python test_ai.py
```

Expected output:

**</> Plain text**

```
PASS: AI takes an immediate winning move.
PASS: AI blocks an immediate opponent win.
PASS: AI handles a board with no legal columns.
All AI tests passed!
```

The test also verifies that the board remains unchanged after the AI evaluates hypothetical moves.

#### 4. Task 3 completion checklist

- ✓ Immediate AI wins take priority.
- ✓ Immediate opponent wins are blocked.
- ✓ Full columns are never selected.
- ✓ Returns `None` when no legal columns exist.
- ✓ Centre preference with random tie-breaking.
- ✓ Hypothetical moves are undone.

- ✓ Existing method signature is preserved.
- ✓ No new dependencies or persistent storage.

**One integration note:** Your Task 2 `game.py` already handles `None` from the AI. Therefore, you can replace `ai.py`, run the tests, and then launch your regular game without changing `game.py`.

#### Task 4 - Move-level feedback

Add concise feedback for a successful disc placement. It should occur once per actual move, not once per cell inspected by win detection or AI analysis.

Current behaviour in `game.py`: after a move the board is just reprinted, with no message saying who played where.

Please add a short message like "You dropped X in column 4." / "AI dropped O in column 5." for both the player and the AI. It should not print for invalid or rejected moves, and it should appear before the win/draw message on the final move. Keep the structure of `game.py`, make the smallest change needed, and explain where you put it and why it only runs once per real move.

Here's the current `game.py`:  
`from board import Board`  
`from ai import AI`

```
class Game:
```

```
 def __init__(self):
```

```
 self.board = Board()
```

```
 self.ai = AI()
```

```
 self.turn = "X"
```

```
 def run(self):
```

```
 print("Connect Four — you are X.")
```

```
 while True:
```

```
 self.board.print()
```

```
 # Handle a full board before requesting another move.
```

```
 if self.board.full():
```

```
 print("Draw! The board is full.")
```

```
 return
```

```
if self.turn == "X":
 raw = input("Column (1-7), or q: ").strip().lower()

 if raw == "q":
 print("Game quit. Thanks for playing!")
 return

 try:
 col = int(raw) - 1
 except ValueError:
 print("Invalid input. Enter a column number from 1 to 7, or q to quit.")
 continue

 # Check the range before attempting to drop a token.
 if not 0 <= col < 7:
 print("Invalid column. Choose a number from 1 to 7.")
 continue

 # Reject full columns without changing the board.
 if self.board.grid[0][col] != ".":
 print(f"Column {col + 1} is full. Choose another column.")
 continue

 else:
 col = self.ai.choose_column(self.board)

 # Handle an AI that cannot find a move.
 if col is None:
 if self.board.full():
 print("Draw! The board is full.")
 else:
 print("AI could not find a move. Game ended.")
 return
```

```
Validate the AI's selected column too.
if not isinstance(col, int) or not 0 <= col < 7:
 print("AI returned an invalid column. Game ended.")
 return

if self.board.grid[0][col] != ".":
 print(f"AI selected full column {col + 1}. Game ended.")
 return

Only valid moves reach this point.
row = self.board.drop(col, self.turn)

Defensive check in case the move is rejected.
if row is None:
 print("Move rejected. Please try again.")
 if self.turn == "O":
 print("AI could not complete its move. Game ended.")
 return
 continue

Check for a win immediately after the move.
if self.board.winner(self.turn):
 self.board.print()
 if self.turn == "X":
 print("You win!")
 else:
 print("AI wins!")
 return

Check for a draw after checking for a win.
if self.board.full():
 self.board.print()
 print("Draw! The board is full.")
 return

Switch turns only when the game is still active.
self.turn = "O" if self.turn == "X" else "X"
```

You only need to add **one print statement** in `game.py`. No other files need to change.

## 1. Make the smallest change

Find this section in your existing `run()` method:

</> Python

```
Defensive check in case the move is rejected.
if row is None:
 print("Move rejected. Please try again.")
 if self.turn == "O":
 print("AI could not complete its move. Game ended.")
 return
 continue
```

Immediately **after this entire** `if row is None` **block**, add:

</> Python

```
Report each successful move exactly once.
if self.turn == "X":
 print(f"You dropped X in column {col + 1}.")
else:
 print(f"AI dropped O in column {col + 1}.")
```

Your updated section should look like this:

Writing



```
```python  
# Defensive check in case the move is rejected.  
if row is None:  
    print("Move rejected. Please try again.")  
    if self.turn == "O":  
        print("AI could not complete its move. Game ended.")  
        return  
    continue
```

```
# Report each successful move exactly once.
if self.turn == "X":
    print(f"You dropped X in column {col + 1}.")
else:
    print(f"AI dropped O in column {col + 1}.")

# Check for a win immediately after the move.
if self.board.winner(self.turn):
    self.board.print()
    if self.turn == "X":
        print("You win!")
    else:
        print("AI wins!")
    return
...
```

Leave the rest of `game.py` unchanged.

2. Why this works

- **Prints once per successful move:** The message appears after `drop()` succeeds, outside the loops that inspect cells during win detection or AI analysis.
- **No message for invalid moves:** Invalid input uses `continue` before reaching this section. Rejected moves where `drop()` returns `None` also skip it.
- **Correct column number:** `col` uses zero-based indexing, so `col + 1` displays the player's familiar column numbers from 1 to 7.
- **Correct order:** The move message prints before the win or draw check, so it also appears before the terminal message on the final move.

3. Example output

</> Plain text

```
You dropped X in column 4.
AI dropped O in column 5.
```

You dropped X in column 4.

You win!

The board display and existing win/draw/quit logic remain unchanged. This completes Task 4 with just a small addition to `game.py`.